

Building Instruction Datasets

GPT-4 generation → MinHash dedup → quality filter → ChatML export → HuggingFace Hub

30 min

Duration

B3 – Apply

Bloom's

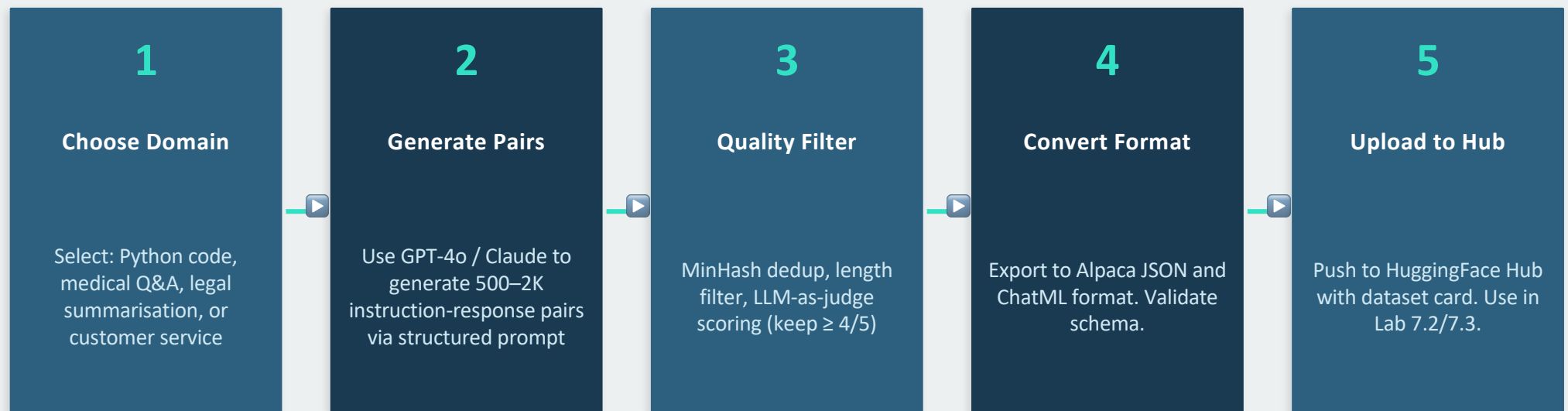
L2 – Intermediate

Level

AI-FT-IN-CD-000004

ID

Lab 7.4 Roadmap



Step 1: Synthetic Data Generation with GPT-4

```
import openai, json, random

client = openai.OpenAI()

SEED_TASKS = [
    "Explain the difference between list and tuple in Python",
    "What is time complexity?",
    "Write a function to reverse a string",
]

def generate_instruction_pair(seed_task):
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{
            "role": "user",
            "content": f"""Generate a diverse, high-quality instruction-
response pair
similar in style to this example: '{seed_task}'

Format your response as JSON:
{{"instruction": "...", "input": "...", "output": "..."}}

Requirements:
- instruction should be a clear, specific task
- output should be accurate, helpful, and 50-300 words
- input can be empty string if not needed"""
        }],
    )
    return json.loads(response.choices[0].message.content)

dataset = [generate_instruction_pair(random.choice(SEED_TASKS)) for _ in
range(500)]
```

Self-Instruct pipeline: use a small seed set → generate many variants → filter for quality. This is how Stanford Alpaca was built.

Step 2: MinHash Deduplication + Quality Filtering

```
from datasketch import MinHash, MinHashLSH
import json

# MinHash LSH deduplication
lsh = MinHashLSH(threshold=0.8, num_perm=128)
deduplicated = []

for i, example in enumerate(dataset):
    mh = MinHash(num_perm=128)
    for word in example["instruction"].lower().split():
        mh.update(word.encode('utf8'))

    if not lsh.query(mh): # no near-duplicates found
        lsh.insert(f"ex_{i}", mh)
        deduplicated.append(example)

print(f"Before dedup: {len(dataset)} | After: {len(deduplicated)}")

# Length filter: remove too short or too long outputs
filtered = [
    ex for ex in deduplicated
    if 30 <= len(ex["output"].split()) <= 500
]

# Convert to ChatML format
def to_chatml(ex):
    return {"conversations": [
        {"from": "human", "value": ex["instruction"] + ("\n" +
ex["input"] if ex["input"] else "")},
        {"from": "gpt", "value": ex["output"]}
    ]}

with open("dataset_chatml.jsonl", "w") as f:
    for ex in filtered:
        f.write(json.dumps(to_chatml(ex)) + "\n")
```

datasketch library: MinHashLSH is $O(n)$ for deduplication — runs on 100K+ examples in seconds. Threshold 0.8 = 80% Jaccard similarity triggers dedup.

KEY REFERENCES

- Wang, Y. et al. (2022). Self-Instruct: Aligning LMs with Self-Generated Instructions. ACL 2023.
- datasketch library. github.com/ekzhu/datasketch
- HuggingFace Hub datasets API. huggingface.co/docs/datasets

 **Next: Concept 19: RLHF Overview (AI-RL-IN-TH-000001)**